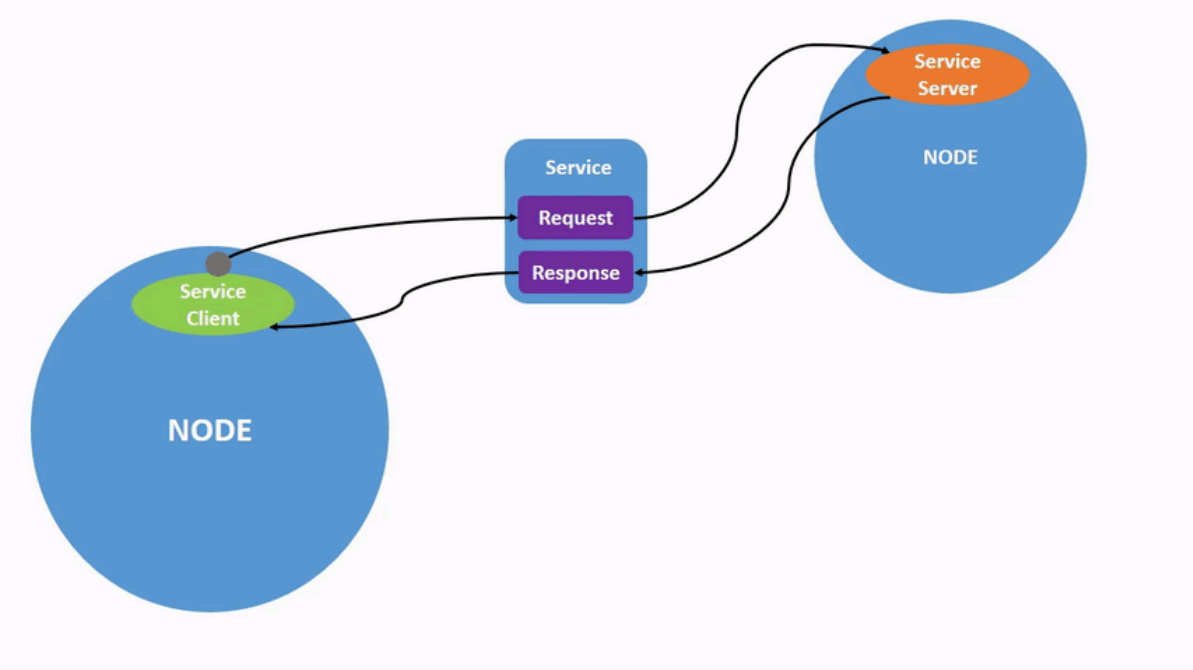
8. ROS 2 Service Communication

1. Introduction to Service Communication

Service communication is a communication model based on request and response. Between the two communicating parties, the client sends request data to the server, and the server responds with the result to the client.

The client/server model is as follows:



From the perspective of the service implementation mechanism, this question-and-answer form is called the client/server model, or CS model for short. When the client needs certain data, it sends a request message for a specific service; after the server receives the request, it processes it and feeds back a response message.

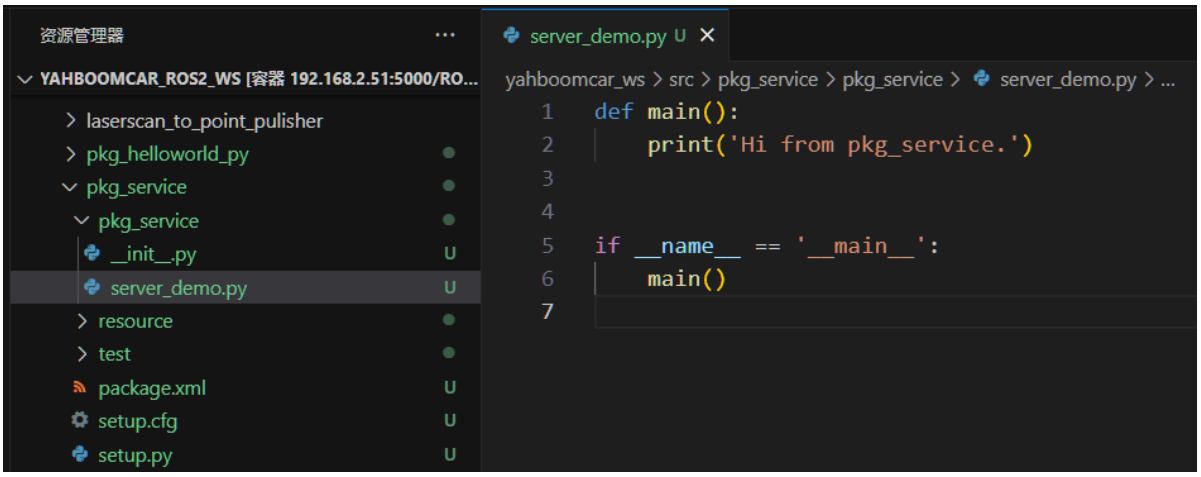
This communication mechanism is also very common in daily life. For example, in the various web pages that are often browsed, your computer browser is the client, which sends a request to the website server through a domain name or various operations, and the server returns the page data to be displayed after receiving it.

2. Create a New Package

- In the src directory of the workspace:

```
ros2 pkg create pkg_service --build-type ament_python --dependencies rclpy --
node-name server_demo
```

After executing the above command, the pkg_service package will be created, and a server_demo node will also be created, with the relevant configuration files already configured.



3. Server Implementation

3.1 Create a Server

Next, edit 【server_demo.py】 to implement the server function and add the following code:

```
#Import the related library files
import rclpy
from rclpy.node import Node
from example_interfaces.srv import AddTwoInts

class Service_Server(Node):
    def __init__(self,name):
        super().__init__(name)
        #Create a server using the create_service function; the parameters passed
in are:
        #The data type of the service data, the service name, and the service
callback function (that is, the content of the service)
        self.srv = self.create_service(AddTwoInts, '/add_two_ints',
self.Add2Ints_callback)
    #The content of the service callback function here is to add two integers and
then return the sum of them
    def Add2Ints_callback(self,request,response):
        response.sum = request.a + request.b
        print("response.sum = ",response.sum)
        return response
def main():
    rclpy.init()
    server_demo = Service_Server("publisher_node")
    rclpy.spin(server_demo)
    server_demo.destroy_node()                    # Destroy the node object
    rclpy.shutdown()                              # Shut down the ROS2 Python
interface
```

Focus on the service callback function, Add2Ints_callback. In addition to self, the parameters that need to be passed in here are request and response. request is the parameter required by the service, and response is the feedback result of the service. request.a and request.b are the contents of the request part, and response.sum is the contents of the response part. First, take a look at what data of the AddTwoInts type looks like.

- You can use the following command to view it

```
ros2 interface show example_interfaces/srv/AddTwoInts
```

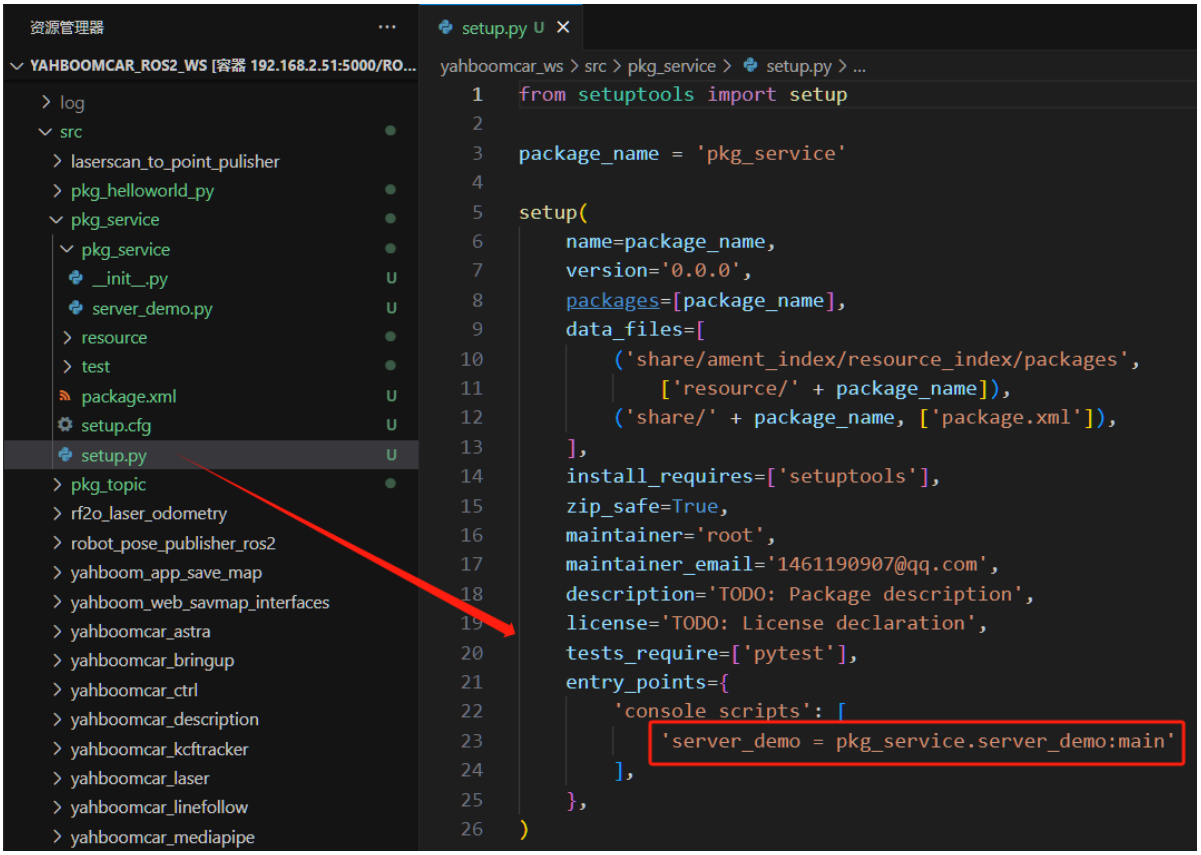
```
root@unbutu:~/yahboomcar_ros2_ws/yahboomcar_ws# ros2 interface show example_interfaces/srv/AddTwoInts
int64 a
int64 b
---
int64 sum
```

"---" divides the data of this type into two parts: the upper part represents request, and the lower part represents response. Each part then has its own variables, such as int64 a and int64 b. Therefore, when the parameters are passed in, the values of a and b need to be specified. Likewise, the feedback result also needs to specify the value of sum.

3.2 Edit the Configuration File

- Open setup.py and add the following to the console_scripts list

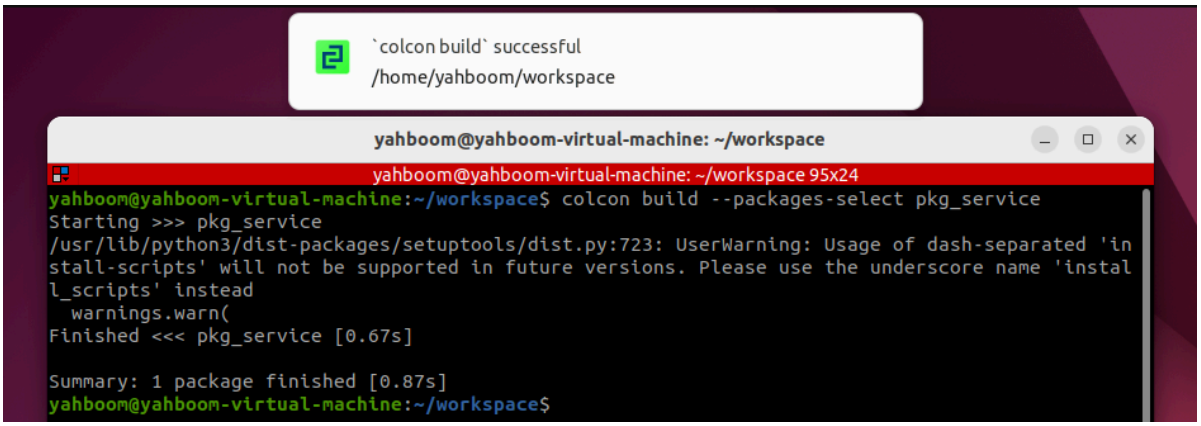
```
'server_demo = pkg_service.server_demo:main',
```



3.3 Compile the Package

- Compile the package

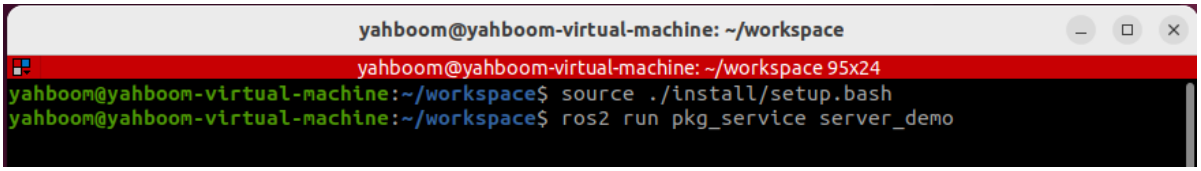
```
colcon build --packages-select pkg_service
```



3.4 Run the Program

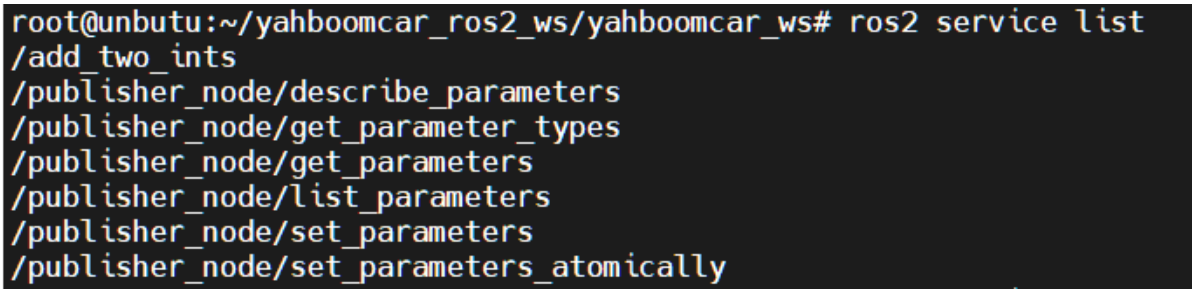
- Refresh the environment variables and then run the node

```
ros2 run pkg_service server_demo
```



After running, since the service is not called, there is no feedback data. The service can be called from the command line. First, query which services currently exist. Enter in another terminal:

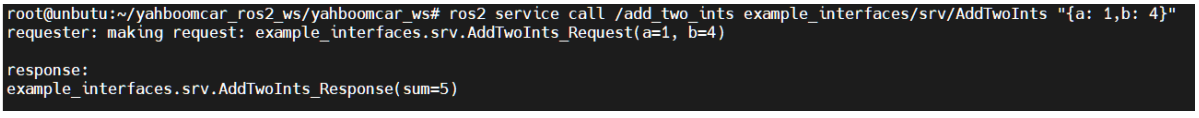
```
ros2 service list
```



/add_two_ints is the service to be called. Call it with the following command. Enter in the terminal:

```
ros2 service call /add_two_ints example_interfaces/srv/AddTwoInts "{a: 1,b: 4}"
```

Here the value of a is assigned 1 and the value of b is assigned 4, that is, the service is called to calculate the sum of 1 and 4:

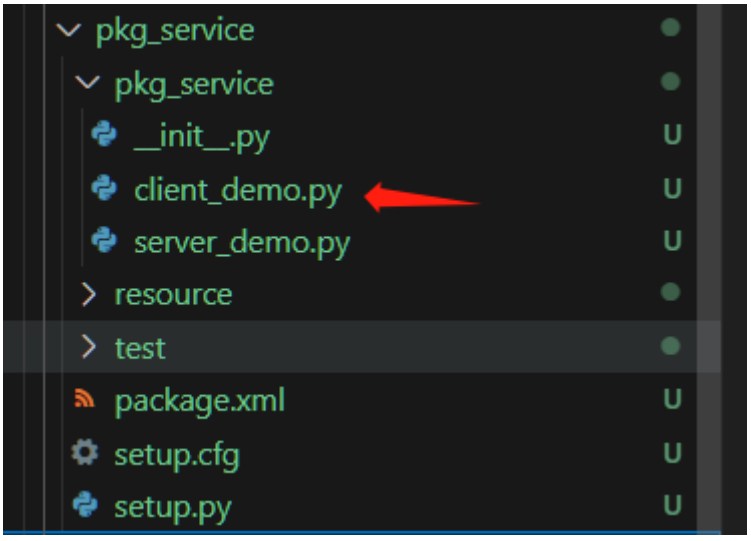


As can be seen from the figure above, after calling the service, the returned result is 5, and the terminal running the server also printed the returned value.

4. Client Implementation

4.1 Create a Client

Create a new file 【client_demo.py】 in the same directory as 【server_demo.py】



Next, edit 【client_demo.py】 to implement the client function and add the following code:

```
#Import the related libraries
import rclpy
from rclpy.node import Node
from example_interfaces.srv import AddTwoInts

class Service_Client(Node):
    def __init__(self,name):
        super().__init__(name)
        #Create a client using the create_client function; the parameters passed
in are the data type of the service data and the topic name of the service
        self.client = self.create_client(AddTwoInts,'/add_two_ints')
        # Loop and wait for the server to start successfully
        while not self.client.wait_for_service(timeout_sec=1.0):
            print("service not available, waiting again...")
        # Create the data object for the service request
        self.request = AddTwoInts.Request()

    def send_request(self):
        self.request.a = 10
        self.request.b = 90
        #Send the service request
        self.future = self.client.call_async(self.request)

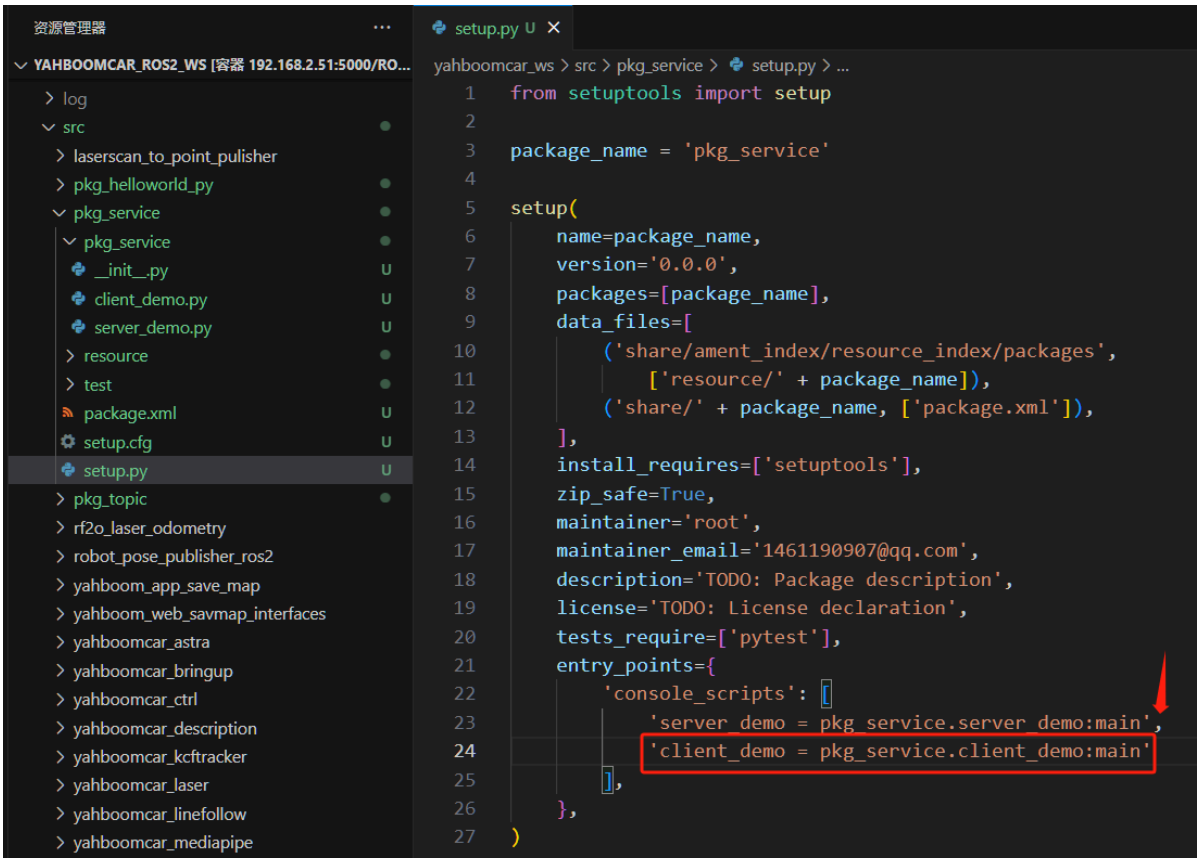
def main():
    rclpy.init() #Node initialization
    service_client = Service_Client("client_node") #Create an object
    service_client.send_request() #Send the service request
    while rclpy.ok():
        rclpy.spin_once(service_client)
        #Determine whether the data processing is complete
        if service_client.future.done():
            try:
```

```
                #Obtain the information fed back by the service and print it
                response = service_client.future.result()
                print("service_client.request.a = ",service_client.request.a)
                print("service_client.request.b = ",service_client.request.b)
                print("Result = ",response.sum)
            except Exception as e:
                service_client.get_logger().info('Service call failed %r' % (e,))
        break
    service_client.destroy_node()
    rclpy.shutdown()
```

4.2 Edit the Configuration File

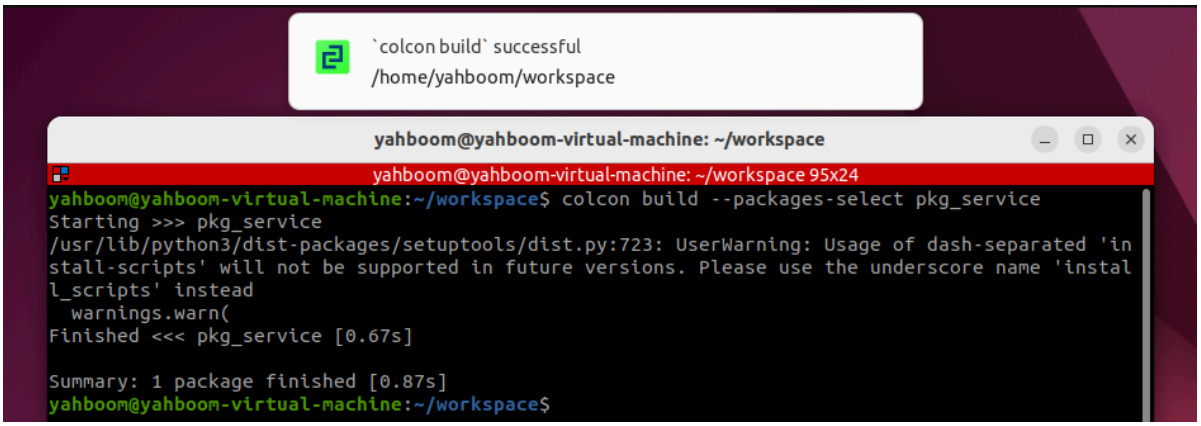
- Open setup.py and add the following to the console_scripts list

```
'client_demo = pkg_service.client_demo:main'
```



4.3 Compile the Package

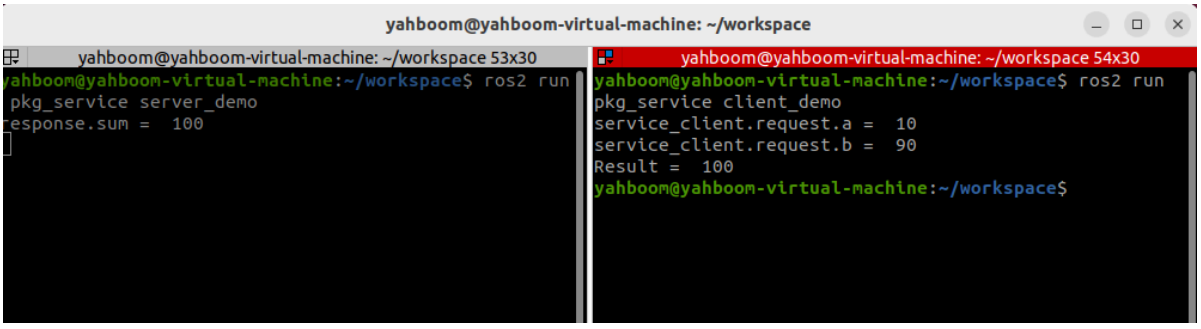
```
colcon build --packages-select pkg_service
```



4.4 Run the Program

- Refresh the environment variables and then run the node

```
#Start the server node
ros2 run pkg_service server_demo
#Start the client node
ros2 run pkg_service client_demo
```



First run the server, then run the client. The client provides a=10 and b=90, the server performs the summation and obtains the result 100, and the result is printed in both terminals.